A system for controlling the interaction of the Rostelecom IT School with universities

English version of ../README.md.

A CRM for the "IT School" case of the "Leaders of Digital Transformation 2026" hackathon. It visualises the multi-stage process of interacting with partner universities and keeps records of contracts, tasks and comments for every interaction.



Prototype

Working prototype: https://crm-rtk.pixel-minds.ru

Demo access:

Role	Login	Password
Administrator	`admin@rtk.ru`	`admin123`
Executive	`manager@rtk.ru`	`manager123`
KAM	`kam@rtk.ru`	`kam123`

What is implemented

- **Kanban board** of interactions with cards moved between stages (`@dnd-kit`), search by university and filtering by product.
- **Interaction card**: details, contract, tasks with deadlines, comments, stage change.
- **Reference data**: universities, IT products, IT directions.
- **Workflow configuration**: adding stages, renaming, column colour, order, enabling and disabling.
- **Report**: key indicators and the distribution of interactions across stages.
- **Roles**: manager and administrator (access to reference data and settings only for the administrator).
- **Responsiveness**: on mobile the board shows one column with a stage selector.
- **Integration with LMS and CMS**: the exchange contract, package preview (dry run) and an idempotent import by `contract_number` (mock adapter).
- **AI summary**: a short digest of the card through GigaChat.
- **Sign-in through Keycloak**: Authorization Code Flow with PKCE; local JWT sign-in remains the fallback.

Business-model constraint

A single university can have **no more than 2 active** interactions — enforced on the backend at creation time.

Stack

Layer	Technologies
Backend	Python 3.11, FastAPI, SQLAlchemy 2 (async), JWT + bcrypt
Frontend	React 18, TypeScript, Vite, Ant Design 5, Zustand, @dnd-kit
Database	PostgreSQL 16 (a container or Yandex Managed PostgreSQL) or SQLite for local development
Cache	Redis / KeyDB for report aggregates
File storage	Yandex Object Storage or MinIO (S3-compatible)
Deployment	the main contour is Yandex Cloud (VM + nginx); the alternative is Render + Vercel + Supabase

Business model

```
University  ─┐
             ├─► Interaction ─► WorkflowStage
ITProduct   ─┘          │
ITDirection ─ ITProduct       ├─ Action (task with a deadline)
                              └─ Comment
```

`University`, `ITProduct`, `ITDirection` are reference data. `Interaction` is the main entity, the "university + product" pair, moving through the stages.

Repository structure

```
backend/          FastAPI application (package backend)
  core/config.py    settings from the environment
  db/               database and session
  models/           models and enums
  schemas/          Pydantic schemas
  api/              routers: auth, universities, directories,
                    stages, interactions, reports
  seed.py           reference data + demo data
  certs/            the Ministry of Digital Development CA root certificate for TLS to GigaChat
  start.sh          starting without Docker (Render, Start Command)
frontend/         React application (Vite)
  src/api/          the axios client and the endpoint description
  src/pages/        BoardPage, ReportPage, DirectoryPage,
                    SettingsPage, LoginPage
  src/components/   MainLayout, kanban, interaction
infra/yandex-cloud/ the production stack for the VM: setup-vm.sh, docker-compose, nginx
render.yaml       Blueprint for the Render + Vercel + Supabase contour
docs/             DEPLOYMENT.md, STACK.md, USER_GUIDE.md,
                  architecture/, security/, pitch.md
```

Quick start through Docker Compose

The whole application comes up with one command. Only Docker with the Compose plugin is needed — no Python or Node on the machine.

```
git clone https://github.com/ForgeLabHub/ForgeRTKCRM.git rtk-crm
cd rtk-crm
cp .env.example .env
# fill in .env — see below
docker compose up -d --build
```

Mandatory values in `.env`:

Variable	How to get it
`SECRET_KEY`	`openssl rand -hex 32`
`MINIO_ROOT_PASSWORD`	invent one (the file storage password)
`KEYCLOAK_ADMIN_PASSWORD`	invent one (the Keycloak administrator)
`DATABASE_URL`	SQLite by default — leave as is for the container

The remaining variables have working defaults. Without a GigaChat key the application still starts: the "Summary" tab will report that the service is not configured.

Readiness check:

```
make doctor        # the environment before start-up: tools, .env, ports
docker compose ps
curl http://localhost:8000/healthz
curl http://localhost:8000/readyz
make smoke-test    # healthz, readyz, /docs, /openapi.json, the frontend
```

Development commands

Command	What it does
`make doctor`	checks tools, versions, `.env`, ports; changes nothing
`make test`	pytest on SQLite, no external database needed
`make lint typecheck`	ruff + mypy
`make alembic-check`	models and migrations are consistent
`make compose-up` / `make compose-down`	the stack; `SERVICES="backend frontend"` for the minimum
`make docs-check`	documentation completeness and module specifications
`make verify`	everything above at once

The "claim → command → CI" matrix: docs/verification/matrix.md. The documentation entry point: ../docs/index.md.

Addresses after start-up:

Service	Address
Frontend	http://localhost:3000
Backend / Swagger UI	http://localhost:8000/docs
Keycloak	http://localhost:8080
MinIO Console	http://localhost:9001
Nginx	http://localhost

> **Swagger UI and `/docs` on a domain.** Swagger UI is served by the backend itself at `/docs` — locally that is `http://localhost:8000/docs`. On a public domain the `/docs` path is taken by the frontend's static documentation (`frontend/public/docs/`), so Swagger UI lives under the same address as the API: `https://crm-rtk.pixel-minds.ru` + `/api` (see `frontend/nginx.conf`).

The database schema and the reference data are created by the application itself at start-up (lifespan): migrations and seeding need no manual run. Demo data is enabled by the `SEED_DEMO_DATA` variable (`true` by default).

To stop: `docker compose down`. To remove everything including the data: `docker compose down -v`.

A Docker-free alternative is the "Local run" section below. A ready interactive script with a menu: `./run.sh start` (or `run.bat start` on Windows).

Local run

Backend

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt
# create .env in the repository root:
# DATABASE_URL=sqlite+aiosqlite:///./dev.db
python scripts\run_dev.py
```

`Settings` reads `.env` from the current directory, so the file goes into the repository root (next to `README.md`), not into `backend/`.

It must be started through `scripts/run_dev.py`: the application lives as the package `app` (`/app` in the container, `backend/` in the repository), so `uvicorn backend.main:app` from the root fails with `ModuleNotFoundError: app`. The script makes the package importable and brings up uvicorn with auto-reload.

Swagger: http://127.0.0.1:8000/docs

Frontend

```
cd frontend
npm install
npm run dev
```

http://127.0.0.1:5173 — the dev server proxies `/api` to `localhost:8000`, so `VITE_API_URL` is not needed in development.

Demo access

Role	E-mail	Password
Administrator	`admin@rtk.ru`	`admin123`
Manager	`manager@rtk.ru`	`manager123`
KAM	`kam@rtk.ru`	`kam123`

Sign-in and authentication

The system supports two modes, switched by `AUTH_MODE` on the backend and `VITE_AUTH_MODE` on the frontend.

Local JWT (`jwt`, the default) — sign-in with the e-mail and password from the form. Suitable for a demonstration and for running without Keycloak.

Keycloak (`keycloak`) — the **Sign in with Keycloak** button on the sign-in page. Authorization Code Flow with PKCE (`S256`) is used: a public client without a client secret, so the authorization code is useless to an interceptor. The backend verifies the RS256 token against the realm's JWKS and takes the roles from the claims.

If Keycloak is unavailable, the sign-in page is not empty: the frontend checks the availability of the server and shows the e-mail/password form as a fallback path.

Details — in docs/KEYCLOAK.md.

Deployment

Two contours are supported; the code is the same, only the environment variables differ.

Yandex Cloud (the main one). One VM, Docker Compose with five containers (PostgreSQL, KeyDB, backend, frontend, nginx). The data is stored in Russia — this is the target contour under 152-FZ and FSTEC order No. 117.

```
docker compose -f infra/yandex-cloud/docker-compose.prod.yml --env-file .env up -d
```

Step by step — in infra/yandex-cloud/README.md and docs/DEPLOYMENT.md.

Render + Vercel + Supabase (the alternative). The backend is a container on Render, the frontend is static files on Vercel, the database is Supabase PostgreSQL. Suitable for a quick public stand without infrastructure of your own.

Documentation

- docs/index.md — the entry point and the map of all documents
- docs/rtk-crm-documentation.pdf — a single PDF with all the documentation
- docs/pdf/ru/ — every document as a PDF (English versions in docs/pdf/en/)
- pitch.md — the presentation script, 11 slides following the LCT template (the text version of `RTK-CRM-LCT2026.pptx`; the deck itself is assembled by the organising committee from this script and is not stored in the repository)
- DEPLOYMENT.md — deployment
- ARCHITECTURE.md — architecture
- KEYCLOAK.md — sign-in through Keycloak and local JWT
- AI.md — the interaction summary (GigaChat)

- USER_GUIDE.md — user guide
- ADMIN_GUIDE.md — administrator guide
- SECURITY.md — security, 152-FZ and privacy
- pitch.md — the presentation script
- qa-jury.md — jury questions and answers
- docs/pdf/ — every document as a PDF in Russian and English

Interface

Screenshots of the current build are in docs/images.

Sign-in

Sign-in

Interaction board (Kanban)

Interaction board

University card

University card

Board filters

Board filters

Reference data

Reference data

XLSX import and field mapping

XLSX import

Reports with chart export to PNG/PDF

Reports

Workflow constructor

Workflow constructor

Users (administrator)

Users

Audit log (administrator)

Audit log

Integration with LMS and CMS

The tab describes the exchange with the customer's external systems: the exchange contract, the package preview (dry run) and an idempotent import by `contract_number`.

Integration with LMS and CMS

Built-in documentation `/help`

Documentation /help

Production

- The main contour: Yandex Cloud, `https://<vm-domain>` (see docs/DEPLOYMENT.md)
- The alternative contour: Render + Vercel + Supabase

Load testing

```
locust -f tests/load/locustfile.py RTKUser \
  --headless -u 50 -r 5 -t 60s \
  --host=http://localhost:8000 \
  --html=tests/load/report.html
```

To test ten parallel reports separately, use the `ReportsUser` scenario; the details are in tests/load/README.md.

License

The project is distributed under the MIT licence.

Project management: `forge.sh` and `forge.bat`

One script for Unix-like systems (`forge.sh`) and one for Windows (`forge.bat`). The interface is identical: the same commands, the same logic, one implementation per platform — no bash scripts inside Windows.

```
./forge.sh              # interactive menu
./forge.sh doctor       # check the environment, changing nothing
./forge.sh install      # install the dependencies
./forge.sh start        # start in the background, wait for readiness, open the browser
./forge.sh status       # what is running and whether the port answers
./forge.sh logs api -f  # logs in real time
./forge.sh test         # tests
./forge.sh stop         # stop
```

```
forge.bat
forge.bat start
forge.bat status
```

What it can do

Command	Action
`status`	state by ports and HTTP, not "by eye"
`start [--fg]`	background (PID files in `.run/`) or foreground
`stop`	stops the whole process tree, the port is released
`restart`, `logs [name] [-f]`, `open`	the usual operations
`doctor`	tools, versions, `.env`, `.env` in git, port occupancy
`install`, `test`, `lint`, `build`, `verify`	through `make`, if it exists
`migrate`	apply the migrations
`db migrate|backup|restore`	the database: migrations, backup, restore
`docker up|down|logs|reset`	containers (`down` does not delete data)
`clean [--hard]`	artefacts and logs; `--hard` also `.venv` and `node_modules`

Rules built into the script

- **It changes nothing without a command.** `doctor` and `status` only read.
- **Data is never deleted.** `stop` and `docker down` keep the database; deletion is only `--hard` or an explicit confirmation in `docker reset`.
- **Pre-flight checks.** If `go`/python is missing or a port is taken, the script says so immediately, not after a two-minute timeout.
- **It does not hang after a crash.** If the process dies, start-up stops and shows the last lines of the log.
- **The venv wins over the system.** If `.venv` exists, `make`, `pytest` and uvicorn use it — so that `install` and `test` do not diverge.

The current way to run without the script: `python scripts/run_dev.py` — the address is `http://localhost:8000/docs`.

Configuration: `.env`

The script reads `.env` **with its own parser**, not through `source`: the file comes from the repository, and arbitrary code must not be executed from it. Supported are `#` comments, empty lines, the `export` prefix, single and double quotes, CRLF, spaces around `=` and a comment after a value.

Value priority: **environment variable → `.env` → built-in value**. So `PORT=9000 ./forge.sh start` works regardless of the file.

Command	Action
`env` / `env show`	the variables, the source of each value and masked secrets
`env init [--force]`	create `.env` from `.env.example`: secret placeholders are replaced with random values, permissions `600`, the previous file goes to `.env.bak`
`env set KEY=VALUE`	write or replace a single variable (atomically, through a temporary file)
`env check`	compare with `.env.example`: missing variables, empty and "sample" secrets, `.env` in `.gitignore`
`env edit`	open the file in `$EDITOR` (fallback: nano → vim → vi)
`env path`, `env reload`	the path to the file and a re-read without a restart

Secrets are masked in the output: `ADMIN_CODE=ch…de (31 chars)`, the password inside a connection string — `postgres://user:***@host`. Names like `*_MINUTES`, `*_URL`, `*_PORT` are not treated as secrets: masking a number of minutes would be pointless.

The path to the file can be overridden: `FORGE_ENV_FILE=/etc/app.env ./forge.sh status`. The Windows version has the same commands: `forge.bat env init`, `forge.bat env check`. A `cmd.exe` limitation: values containing `%` and `!` are not supported.

Flags and completion

```
./forge.sh -h | --help           # help
./forge.sh -V | --version        # the script version, the commit, the state of .env
./forge.sh -q status             # say nothing about successes
./forge.sh --no-color status     # no ANSI (the same as NO_COLOR=1)
./forge.sh --json status         # machine output for scripts and monitoring

eval "$(./forge.sh completion bash)"   # bash completion
./forge.sh completion zsh > ~/.zfunc/_forge   # for zsh
```

`test`, `lint`, `build` and `verify` run in an environment **without** the variables from `.env`: otherwise local checks would behave differently from CI, and tests like "without a key there must be a fallback" would stop verifying anything.

Running in this project

```
./forge.sh install && ./forge.sh start
```